

VIETNAM NATIONAL UNIVERSITY – HO CHI MINH CITY  
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY  
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



## **Specialized Project – Artificial Intelligence**

**HK261-DAGD1-041**

---

### **Technical Report**

# **An End-to-End MLOps Pipeline for Cross-Sectional Equity Ranking under Data Constraints: Engineering Validation and Ablation Study.**

---

|              |                            |            |
|--------------|----------------------------|------------|
| Supervisors: | Dr. Pham An Vinh, Ph.D     | ID 004361  |
|              | Dr. Nguyen An Khuong, Ph.D | ID 003634  |
| Students:    | Doan Phuong Hung Cuong     | ID 2310381 |
|              | Nguyen Thien Hao           | ID 2310853 |
|              | Pham Quang Hieu            | ID 2310974 |

HO CHI MINH CITY, 12 SEPTEMBER 2026



# Declaration

We, the undersigned – Doan Phuong Hung Cuong, Nguyen Thien Hao, and Pham Quang Hieu – hereby submit this Specialized Project Report, titled “*An End-to-End MLOps Pipeline for Cross-Sectional Equity Ranking under Data Constraints: Engineering Validation and Ablation Study*”, in partial fulfillment of the academic requirements. We solemnly declare that this manuscript represents the authentic outcome of our independent research, system architecture design, and empirical validation, conducted under the supervision of **Dr. Pham An Vinh** and **Dr. Nguyen An Khuong**.

We explicitly affirm the following statements regarding academic integrity, ethical standards, and original authorship:

- **Originality and Intellectual Attribution:** All methodologies, algorithms, data sources, and literature referenced herein that are not our original creation have been explicitly acknowledged, formally cited, and referenced in strict compliance with established academic standards.
  - **Disclosure of AI and LLMs Assistance:** Generative Artificial Intelligence (AI) systems and Large Language Models (LLMs) were utilized strictly as supportive tools for syntactic refinement, prose editing, and structural organization. No core technical architectures, empirical analyses, feature engineering logic, or conclusions were synthesized by automated tools. The authors retain full ownership of and accountability for all intellectual content.
  - **Data Integrity and Empirical Veracity:** All computational experiments, performance metrics, and ablation outcomes presented in this work reflect genuine implementation results without fabrication, falsification, or deliberate omission of data.
  - **Exclusivity of Submission:** This research work is original and has not been submitted, either in whole or in part, for the award of any other academic degree, diploma, or professional qualification at this or any other institution.
-



# Acknowledgment

First and foremost, we extend our deepest gratitude to [Dr. Pham An Vinh](#) for his exceptional technical mentorship and visionary guidance throughout this project. His profound expertise in quantitative finance and practical insights into exploitable alpha methodologies served as the absolute cornerstone of our research. His extensive real-world experience empowered us to successfully navigate the profound complexities inherent in modeling intricate market architectures, identifying persistent statistical trends, and analyzing market microstructure dynamics. Furthermore, his rigorous oversight significantly elevated the mathematical precision of our quantitative analysis, particularly in the robust empirical evaluation, feature engineering, and advanced backtesting of our cross-sectional equity ranking pipelines.

We also wish to express our sincere appreciation to our supervisor, [Dr. Nguyen An Khuong](#). His invaluable support and steadfast academic guidance have been instrumental in refining our technical acumen and instilling a rigorous, systematic approach to both our research and engineering efforts. We are especially thankful for his generous provision of foundational academic materials and specialized literature, which proved critical to the successful formulation, structural organization, and execution of our research objectives.

Our sincere thanks are directed to the Faculty of Computer Science and Engineering at [Ho Chi Minh City University of Technology](#)  for fostering a stimulating academic environment and supplying the foundational resources requisite for conducting this specialized study.

Furthermore, we gratefully acknowledge [Modal](#)  for their generous provision of cloud computing infrastructure. Access to their high-performance, complimentary Graphics Processing Unit resources was pivotal in overcoming our local hardware limitations, enabling the efficient execution of complex computational experiments, large-scale ablation studies, and the training of our machine learning pipelines.

We also wish to recognize our peers and colleagues whose intellectual camaraderie, constructive feedback, and steadfast encouragement provided much-needed motivation during the more demanding phases of this project.

Finally, we are eternally grateful to our families for their unwavering emotional support, patience, and understanding throughout the extensive development, implementation, and writing of this report.

---



# Abstract



# Contents

|            |                                                                        |          |
|------------|------------------------------------------------------------------------|----------|
| <b>I</b>   | <b>Introduction</b>                                                    | <b>1</b> |
| 1          | Context and Motivation                                                 | 1        |
| 2          | Research Problem                                                       | 2        |
| 3          | Research Gap                                                           | 3        |
| 4          | Research Questions                                                     | 4        |
| 5          | Research Contributions                                                 | 4        |
| 6          | Thesis Organization                                                    | 5        |
| 7          | Overall Structure                                                      | 5        |
| <b>II</b>  | <b>Methodology</b>                                                     | <b>6</b> |
| 8          | Problem Formulation                                                    | 6        |
| 9          | Target Construction                                                    | 7        |
| 10         | Project Objectives                                                     | 8        |
| <b>III</b> | <b>Market Reconstruction</b>                                           | <b>9</b> |
| 11         | Market Data Acquisition and Synthesis Pipeline                         | 10       |
| 11.1       | Pipeline Architecture and Data Flow . . . . .                          | 10       |
| 11.2       | Historical Trading Calendar and Active-Ticker Reconstruction . . . . . | 11       |
| 11.3       | Distributed Acquisition and Persistent Storage . . . . .               | 14       |
| 11.4       | Ticker Availability Reconstruction . . . . .                           | 17       |
| 11.5       | Security-Level OHLCV Acquisition . . . . .                             | 17       |
| 12         | Data Integrity                                                         | 18       |
| 12.1       | Data Quality and Missing Observations . . . . .                        | 18       |
| 13         | Factor Data                                                            | 18       |



|                                                             |           |
|-------------------------------------------------------------|-----------|
| <b>14 Point-in-Time Universe Construction</b>               | <b>18</b> |
| <b>IV Data Transformation</b>                               | <b>19</b> |
| 14.1 Feature Engineering . . . . .                          | 19        |
| 14.1.1 Logarithmic Returns . . . . .                        | 19        |
| 14.1.2 Distance from Simple Moving Average (DSMA) . . . . . | 19        |
| 14.1.3 Relative Strength Index (RSI) . . . . .              | 20        |
| 14.1.4 Relative Volume (RVOL) . . . . .                     | 21        |
| 14.1.5 Ichimoku Indicator . . . . .                         | 21        |
| 14.1.6 Price Volume Trend (PVT) . . . . .                   | 23        |
| 14.1.7 Rolling Volatility . . . . .                         | 23        |
| 14.2 Feature Extraction . . . . .                           | 24        |
| 14.3 Feature Selection . . . . .                            | 24        |
| <b>V Alpha Implementation</b>                               | <b>25</b> |
| <b>15 Model Architecture</b>                                | <b>25</b> |
| 15.1 Temporal Encoding (LSTM) . . . . .                     | 25        |
| 15.2 Context Layer (Cross-Sectional Transformer) . . . . .  | 25        |
| 15.3 Hybrid Loss Function . . . . .                         | 25        |
| <b>16 Hyperparameter Optimization</b>                       | <b>25</b> |
| <b>17 Training Procedure</b>                                | <b>25</b> |
| <b>VI Empirical Results</b>                                 | <b>26</b> |
| <b>18 Descriptive Statistics and Data Diagnostics</b>       | <b>27</b> |
| 18.1 Dataset Characteristics . . . . .                      | 27        |
| 18.2 Target and Feature Diagnostics . . . . .               | 27        |
| <b>19 Predictive Performance</b>                            | <b>27</b> |
| 19.1 IC/ICIR . . . . .                                      | 27        |
| 19.2 Decile Portfolio Monotonicity . . . . .                | 27        |
| 19.3 Kendall's Tau Analysis . . . . .                       | 27        |
| <b>20 Statistical Diagnostics</b>                           | <b>27</b> |
| 20.1 Statistical Significance . . . . .                     | 27        |

---



|                                                             |           |
|-------------------------------------------------------------|-----------|
| 20.2 Multiple-Testing and Overfitting Diagnostics . . . . . | 27        |
| <b>21 Ablation Results</b>                                  | <b>27</b> |
| 21.1 Model Architecture Ablation . . . . .                  | 27        |
| 21.2 Loss Function Ablation . . . . .                       | 27        |
| 21.3 Stock ID Embedding Ablation . . . . .                  | 27        |
| <b>22 Explainability and Regime-Conditioned Analysis</b>    | <b>27</b> |
| 22.1 Integrated Gradients Attribution . . . . .             | 27        |
| 22.2 Feature Importance Across Volatility Regimes . . . . . | 27        |
| 22.3 Surrogate Model Fidelity . . . . .                     | 27        |
| 22.4 Interpretation of Model Behavior . . . . .             | 27        |
| <b>VII Portfolio Construction</b>                           | <b>28</b> |
| <b>VIII Appendixes</b>                                      | <b>29</b> |

---



## List of Figures

|                                                                  |    |
|------------------------------------------------------------------|----|
| 10.1 Survivorship Bias Illustration . . . . .                    | 9  |
| 11.1 Market Data Acquisition & Reconstruction Pipeline . . . . . | 11 |
| 11.2 Workers Distributed Architecture . . . . .                  | 14 |

## List of Tables

|                                                    |    |
|----------------------------------------------------|----|
| 11.1 NYSE Trading Calendar Configuration . . . . . | 12 |
| 11.2 Parallel NYSE Worker Splits. . . . .          | 15 |

## List of Abbreviations

|              |                                           |
|--------------|-------------------------------------------|
| <b>AI</b>    | Artificial Intelligence                   |
| <b>API</b>   | Application Programming Interface         |
| <b>DSMA</b>  | Distance from Simple Moving Average       |
| <b>GPU</b>   | Graphics Processing Unit                  |
| <b>IC</b>    | Information Coefficient                   |
| <b>ICIR</b>  | Information Coefficient Information Ratio |
| <b>LLMs</b>  | Large Language Models                     |
| <b>LSTM</b>  | Long Short-Term Memory                    |
| <b>MLOps</b> | Machine Learning Operations               |
| <b>NYSE</b>  | New York Stocks Exchange                  |
| <b>OHLCV</b> | Open-High-Low-Close-Volume                |
| <b>PVT</b>   | Price Volume Trend                        |
| <b>RSI</b>   | Relative Strength Index                   |

---



## Part I

# Introduction

## 1 Context and Motivation

Machine learning has become an increasingly important methodological tool in quantitative finance because of its ability to model nonlinear relationships and interactions among large sets of financial predictors. In a large-scale empirical study of equity returns, Gu, Kelly, and Xiu<sup>[9]</sup> show that machine learning methods can improve return forecasting relative to conventional linear approaches, with part of the predictive improvement arising from nonlinear interactions among characteristics. Their results also identify momentum, liquidity, and volatility-related variables among the dominant predictors of expected returns. This evidence motivates the use of flexible nonlinear models when the objective is to extract information from a rich set of financial signals.

Sequence-based deep learning models provide one approach to capturing temporal structure in financial data. Fischer and Krauss<sup>[8]</sup> demonstrate the application of Long Short-Term Memory (LSTM) networks to financial market prediction and report that LSTM models can exploit patterns in sequential stock data that are difficult to capture using simpler predictive models. More generally, the Transformer architecture<sup>[12]</sup> provides a mechanism for modeling relationships through attention, allowing a model to selectively incorporate information from multiple elements of an input sequence. These characteristics make recurrent and attention-based architectures potentially suitable for financial problems in which relevant information may be distributed across time and across simultaneously observed securities.

The present study focuses specifically on the latter dimension. The objective is not to forecast the return of an individual stock in isolation, but to generate a *cross-sectional ranking* of securities at each decision point. This distinction is fundamental because the economic decision in a cross-sectional strategy is based on the relative ordering of securities rather than the numerical accuracy of an isolated return forecast. Poh et al.<sup>[10]</sup> formalize this perspective and show that ranking-oriented learning methods can improve cross-sectional systematic strategies by directly learning pairwise and listwise relationships among instruments. The ranking problem therefore requires a modeling framework that preserves information about the contemporaneous cross-section instead of treating each security as an independent forecasting task.

The cross-sectional nature of equity returns is also well established in the asset-pricing literature. Fama and French<sup>[6]</sup> show that firm characteristics such as size and book-to-market equity are related to systematic differences in average stock returns, while Fama and French<sup>[7]</sup> demonstrate that common risk factors account for substantial shared variation in stock returns.

These findings motivate a distinction between systematic variation shared across securities and residual variation that may contain additional information about relative performance. In the present study, this distinction is reflected in the construction of a forward-return target that is residualized against common risk factors before cross-sectional evaluation.

The research problem is therefore not simply to determine whether a flexible model can predict future stock returns. Rather, it is to determine whether temporal information and cross-sectional relationships can be combined to improve the ranking of securities within a common market universe. The proposed architecture addresses this problem by combining an LSTM-based temporal encoder with a cross-sectional Transformer, allowing information about the historical evolution of an individual security to be processed jointly with information from the contemporaneous cross-section.

## 2 Research Problem

The central research problem addressed in this thesis is the design and empirical validation of an end-to-end machine learning pipeline for cross-sectional equity ranking under the statistical, data, and implementation constraints inherent in financial markets.

A conventional pointwise forecasting formulation assigns a prediction problem to each security separately. However, the investment decision considered in this study is inherently comparative: securities are ranked relative to their peers and subsequently incorporated into a portfolio according to their predicted relative performance. The learning objective must therefore reflect the structure of the downstream decision problem. This motivates the use of a ranking-aware learning objective in addition to the conventional regression objective, following the cross-sectional learning-to-rank perspective of Poh et al.<sup>[10]</sup>.

At the same time, the predictive problem must account for common systematic variation. Because securities in the same market universe are exposed to shared market conditions, raw forward returns contain variation that may not represent security-specific information. The empirical design therefore residualizes the forward return against common factors before evaluating cross-sectional predictive performance. This approach is consistent with the role of common risk factors established in the asset-pricing literature<sup>[6;7]</sup>.

A second dimension of the research problem concerns empirical validation. Financial observations are temporally ordered and may contain overlapping information across adjacent observations. Consequently, model evaluation must preserve chronological structure and distinguish information available at the prediction date from information that becomes available subsequently. Tashman<sup>[11]</sup> emphasizes the importance of carefully designed out-of-sample evaluation and rolling-origin procedures in forecasting applications. In addition, extensive hyperparameter

experimentation creates a multiple-testing problem: a model selected because it performs well over many trials may exhibit an inflated backtest performance even when the underlying strategy contains little genuine predictive information. Bailey and López de Prado<sup>[2]</sup> develop the Deflated Sharpe Ratio specifically to address selection bias and backtest overfitting arising from multiple testing.

The research therefore treats validation as part of the model itself rather than as a separate reporting step. The proposed pipeline incorporates chronological walk-forward evaluation, purging and embargoing of observations around test periods, and explicit adjustment for multiple-testing effects. These procedures are intended to reduce the possibility that apparent predictive performance results from information leakage or repeated experimentation rather than from generalizable structure.

Finally, the research considers the transition from statistical prediction to portfolio construction. A cross-sectional ranking may contain predictive information without being directly implementable as a portfolio. Portfolio decisions are affected by covariance estimation, position constraints, turnover, transaction costs, and market impact. The framework consequently evaluates the model's out-of-sample predictions within a constrained portfolio construction process. The importance of accounting for execution costs and market impact in portfolio implementation is established in the optimal-execution literature<sup>[1]</sup>.

### 3 Research Gap

The first research gap concerns the alignment between prediction objectives and cross-sectional portfolio decisions. Standard regression-based forecasting evaluates the accuracy of individual return predictions, whereas cross-sectional strategies ultimately depend on the relative ranking of securities. Poh et al.<sup>[10]</sup> demonstrate the importance of learning the ranking structure directly. The present study extends this perspective by investigating a hybrid learning objective that combines pointwise prediction with pairwise ranking information within a deep temporal–cross-sectional architecture.

The second gap concerns the integration of temporal and cross-sectional information within a single predictive framework. LSTM-based models provide a mechanism for learning sequential dependencies<sup>[8]</sup>, while attention-based models provide a mechanism for representing interactions among elements of an input set<sup>[12]</sup>. Existing empirical work establishes the usefulness of both nonlinear machine learning and cross-sectional structure in equity prediction<sup>[9;10]</sup>; however, the present study focuses on an integrated architecture in which temporal representations of individual securities are subsequently processed jointly across the cross-section.

The third gap concerns validation under realistic financial data constraints. Machine learning

models provide substantial flexibility, but that flexibility also increases exposure to data leakage, selection bias, and backtest overfitting. Tashman<sup>[11]</sup> establishes the importance of carefully designed out-of-sample forecasting procedures, while Bailey and López de Prado<sup>[2]</sup> show that multiple testing can materially inflate apparent strategy performance. The present research incorporates these concerns directly into the experimental design through purged walk-forward validation, embargoing, and multiple-testing diagnostics.

The fourth gap concerns the connection between predictive signals and implementable portfolios. Rather than evaluating the model solely through prediction accuracy, the study carries locked out-of-sample predictions into a constrained portfolio construction stage. This allows the robustness of the predictive signal to be examined after accounting for covariance risk and explicit transaction-cost and market-impact assumptions, consistent with the principles of optimal execution<sup>[1]</sup>.

## 4 Research Questions

The central research question is:

Can an end-to-end MLOps pipeline successfully engineer, validate, and constrain a hybrid temporal-attention (LSTM–Transformer) architecture for cross-sectional equity ranking, and accurately translate its out-of-sample predictions into a strictly risk-neutralized portfolio under rigid computational and data constraints?

This question is investigated through two groups of research questions. The first concerns signal generation and model validation. It examines whether a hybrid loss combining Mean Squared Error (MSE) with a pairwise hinge loss improves the separation of predicted cross-sectional deciles relative to a pure pointwise objective, and whether a learned static Stock ID embedding contributes information beyond the temporal and cross-sectional market structure.

The second group concerns portfolio construction and robustness. It examines the degradation of model-derived portfolio performance after the introduction of covariance risk and transaction costs, and evaluates the extent to which multiple-testing adjustment reduces the apparent performance obtained during model and hyperparameter selection.

## 5 Research Contributions

The contribution of this thesis is primarily methodological and empirical. It develops and evaluates an end-to-end framework for cross-sectional equity ranking in which predictive modeling, statistical validation, and portfolio construction are treated as interconnected components.



First, the study formulates the equity prediction problem explicitly as a cross-sectional ranking task rather than as a set of independent point forecasts. Second, it combines temporal representation learning through an LSTM with cross-sectional attention through a Transformer, thereby allowing both within-security dynamics and contemporaneous inter-security relationships to be modeled. Third, it incorporates a ranking-aware training objective intended to align the optimization problem with the downstream portfolio decision. Fourth, the framework embeds chronological and leakage-aware validation procedures together with explicit control for multiple-testing bias. Finally, it evaluates the persistence of the generated signal after portfolio construction under covariance, leverage, position, turnover, and transaction-cost constraints.

The study is deliberately framed as an engineering validation and empirical evaluation rather than a claim that the resulting model constitutes universally deployable trading alpha. The empirical analysis is conducted within the explicitly defined S&P 500 research universe and its filtered analytical sample, and the methodological limitations associated with the available data are retained and discussed explicitly.

## 6 Thesis Organization

The remainder of the thesis follows the sequence of the empirical pipeline. The methodology first defines the market universe and constructs the analytical sample, including the associated survivorship and classification considerations. The data-processing stage then describes the construction and transformation of the input variables and prediction target. The predictive modeling stage presents the temporal and cross-sectional architecture, the ranking-aware learning objective, and the experimental design. The subsequent evaluation examines predictive performance, statistical significance, model stability, and sensitivity to alternative specifications. Finally, the locked out-of-sample predictions are passed to the portfolio-construction framework, where the economic consequences of risk controls and transaction costs are evaluated before the methodological limitations and conclusions are presented.

## 7 Overall Structure

To be updated

## Part II

# Methodology

## 8 Problem Formulation

### The Cross-Sectional Ranking Objective

The investment decision analyzed in this study is inherently comparative: the goal is to rank securities relative to each other, rather than to predict the absolute numeric accuracy of each individual return. As such, the problem is cast as a *learning-to-rank* task. Denote by  $N$  the number of active equities in the analytical sample at time  $t$ . At each prediction date, the model receives a three-dimensional input tensor of temporal features,  $\mathbf{X}_t \in \mathbb{R}^{N \times T \times F}$ , where  $T$  is the lookback sequence length (e.g., 60 days), and  $F$  is the number of engineered features extracted per security.

The learning objective is to parameterize a mapping  $f(\mathbf{X}_t; \Theta)$  that outputs a vector of cross-sectional scores  $\hat{\mathbf{y}}_t \in \mathbb{R}^N$ . These scores should monotonically align with the realized forward *residualized* returns  $\mathbf{y}_t \in \mathbb{R}^N$ , meaning the induced orderings serve as the ranking for the cross-section at time  $t$ .

### Generalization and Bias Neutralization

A central design constraint is to prevent structural survivorship bias and regime overfitting. Since the sample is composed of continuous survivors from 2020 onward, including static identifiers (such as Stock IDs or persistent entity embeddings) would allow the model to memorize long-term “winners”, thus violating generalizability. To enforce that the model learns a generalized mapping (a pricing kernel),  $f(\mathbf{X}_t; \Theta)$  is constructed to explicitly exclude such static structural identifiers. Both the temporal encoder (LSTM) and the context layer (Cross-Sectional Transformer) must deduce the cross-sectional ranking solely from the time-varying technical and fundamental features present in  $\mathbf{X}_t$ , observed at each  $t$ .

### Hybrid Optimization Function

To align mathematical training with the requirements of downstream portfolio construction, the network is trained with a *hybrid* loss. This combines classical Mean Squared Error (MSE), which helps anchor the scale of the predictions, with a pairwise hinge loss. The hinge component operates on each cross-sectional batch and explicitly penalizes situations where securities from empirically lower-performing quantiles are scored above those from higher-performing

quantiles. This approach directly incentivizes the monotonic separation required for constructing a market-neutral, cross-sectional portfolio.

## 9 Target Construction

### Systematic Risk Residualization

Securities within a broad equity universe share exposure to the same market-wide risk factors; thus, raw forward returns are dominated by common systematic variation that is not stock-specific. To ensure that the model's learning target strictly captures *security-specific* signals, the forward-looking prediction target is residualized with respect to known risk factors prior to cross-sectional evaluation.

### Fama-French 3-Factor Rolling OLS

The idiosyncratic return target for each stock is constructed via the Fama-French 3-Factor model. For every security  $i$ , a trailing 126-day (roughly six months) rolling Ordinary Least Squares (OLS) regression is performed at date  $t$ :

$$R_{i,t} - R_{f,t} = \alpha_{i,t} + \beta_{i,\text{MKT}} \text{MKT}_t + \beta_{i,\text{SMB}} \text{SMB}_t + \beta_{i,\text{HML}} \text{HML}_t + \epsilon_{i,t} \quad (9.1)$$

where  $R_{i,t}$  is the return for stock  $i$ ,  $R_{f,t}$  is the risk-free rate, and  $\text{MKT}_t$ ,  $\text{SMB}_t$ , and  $\text{HML}_t$  denote the market, size, and value factor premia, respectively. The residual  $\epsilon_{i,t}$  isolates the stock-specific, non-systematic return component at each date.

### Cross-Sectional Z-Scoring for the Forward Horizon

To produce the final set of predictive targets  $\mathbf{y}_t$  for each ranking instance, the daily idiosyncratic residuals  $\epsilon_{i,t}$  are aggregated for each security over a 5-day forward prediction horizon. To ensure that the resulting target is scale-invariant and purely comparative, these 5-day aggregated residuals are cross-sectionally Z-scored at each  $t$ . That is, for the vector of  $N$  stocks on a given day, the transformed target has mean 0 and standard deviation 1 across the cross-section:

$$\mathbf{y}_t = \frac{\boldsymbol{\epsilon}_t^{(5)} - \text{mean}(\boldsymbol{\epsilon}_t^{(5)})}{\text{std}(\boldsymbol{\epsilon}_t^{(5)})} \quad (9.2)$$

This enforces that the model's predictions are evaluated exclusively by their relative ordering, consistent with the long/short, market-neutral ranking objective.



## 10 Project Objectives

p

## Part III

# Market Reconstruction

Survivorship bias is a well-established problem in empirical financial research: restricting historical analysis to securities that survive until the end of the sample can produce distorted estimates and misleading evidence of return predictability. Brown et al. <sup>[3]</sup> demonstrate that survivorship-truncated samples can materially affect empirical conclusions.

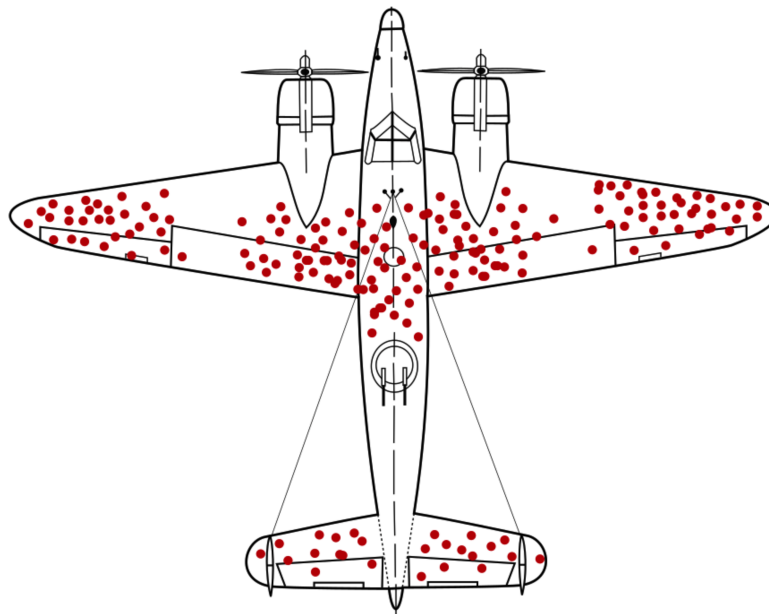


Figure 10.1: Survivorship Bias Illustration

For this study, avoiding survivorship bias is complicated by the limited access provided by free-tier market-data services. No single freely accessible source is assumed to provide both a sufficiently long historical security universe and the complete security-level price data required by the study. Consequently, the dataset must be reconstructed by combining complementary sources rather than directly downloaded as a finished historical panel.

The pipeline therefore uses the New York Stocks Exchange (NYSE) trading calendar as its temporal backbone, retrieves daily historical active-ticker snapshots from [Massive](#) , aggregates these observations with Polars to derive ticker availability intervals, and uses those intervals to obtain daily OHLCV data from Yahoo Finance. This produces a persistent raw market dataset that preserves historical security availability while remaining feasible under the constraints of the available free-tier APIs. The resulting data then serve as the foundation for the subsequent

integrity checks, point-in-time universe construction, data-quality procedures, and machine-learning pipeline.

## 11 Market Data Acquisition and Synthesis Pipeline

The pipeline is designed as a sequential data-reconstruction process, in which each stage produces the information required by the next rather than treating market-data acquisition as a single download operation. The central design principle is to separate historical security availability from security-level market observations. Massive’s ticker-reference endpoint supports querying securities available on a specified date and filtering by active status, making it suitable for constructing the historical availability layer.

### 11.1 Pipeline Architecture and Data Flow

The first stage establishes which dates need to be processed, focusing on trading sessions actually observed on the NYSE rather than merely using calendar days. For each trading session, Massive is queried using the date and the active-stock filter. Its API explicitly supports retrieval of tickers available on any specified date and also returns pagination information when results exceed a single-page, allowing the complete daily snapshot to be reconstructed for each session.

These daily snapshots are deliberately retained before aggregation, creating a two-level historical-universe representation:

1. **Daily point-in-time snapshots**, preserving the original record of which securities were present on each trading date.
2. **Ticker availability intervals**, providing a compact security-level summary derived from these daily observations.

Polars transforms the first representation into the second by grouping all sessions for each ticker and taking the minimum and maximum dates:

$$\text{start\_date}_i = \min(T_i), \quad \text{end\_date}_i = \max(T_i)$$

where  $T_i$  denotes the set of observed trading dates for ticker  $i$ . The resulting availability table serves as the link between universe reconstruction and downstream market-data acquisition.

Next, Yahoo Finance is queried using these reconstructed ticker-specific intervals to download daily OHLCV observations. Importantly, this stage does not determine historical universe membership – that information has already been robustly reconstructed. This separation ensures that the market-data source does not implicitly define the security universe, allowing independent data integrity checks and point-in-time investment universe construction in later sections.

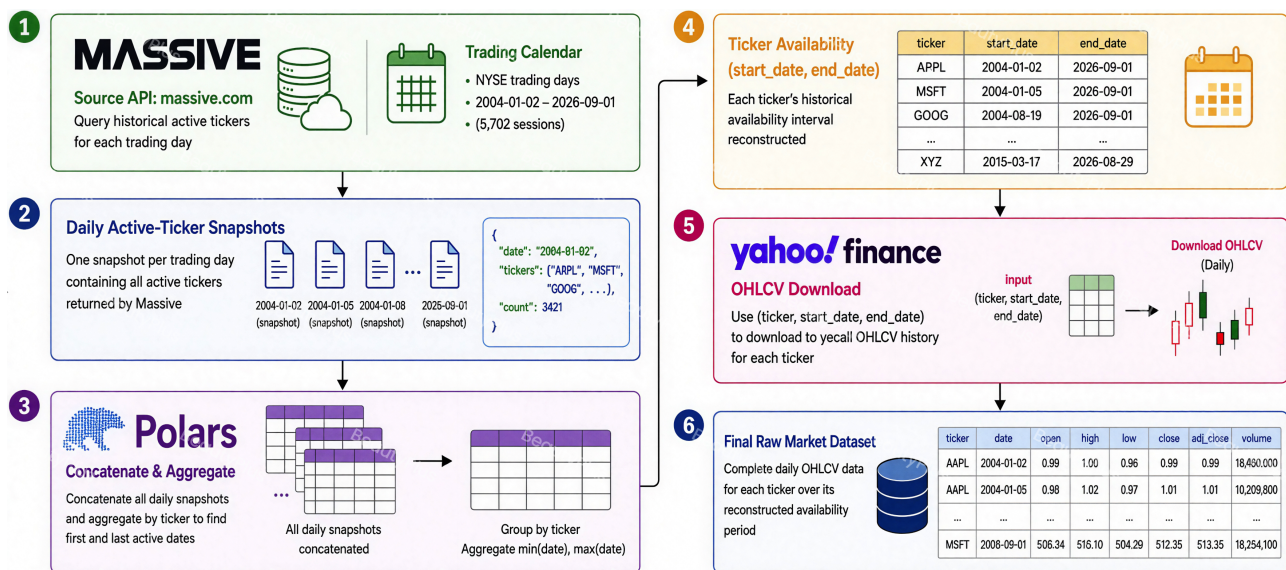


Figure 11.1: Market Data Acquisition & Reconstruction Pipeline

Thus, the pipeline architecture can be conceptually decomposed into two connected but distinct processes:

### Market-data acquisition:

*Ticker Availability → Yahoo Finance → Raw OHLCV Dataset*

### Historical availability reconstruction:

*NYSE Calendar → Massive → Daily Snapshots → Polars → Ticker Availability*

This functional separation is especially important given the constraints of free-tier data access. Instead of requiring a single source to provide a complete, survivorship-aware market panel, the pipeline distributes the reconstruction effort across complementary data providers and preserves all intermediate representations necessary for reproducibility and future audit. Notably, Massive's documentation affirms that its reference-ticker service supports historical, date-specific active-status queries – the crucial capability leveraged in the first stages of the pipeline.

## 11.2 Historical Trading Calendar and Active-Ticker Reconstruction

The first stage establishes the **historical temporal and security universe** over which the remainder of the pipeline operates. Rather than iterating over calendar days, the pipeline first generates the official NYSE trading sessions for the study period and then queries the historical active-ticker universe for each session. This ensures that the reconstruction is aligned with actual market activity rather than an arbitrary sequence of calendar dates. `pandas_market_calendars` provides exchange-specific calendars, including NYSE holidays, regular sessions, and special market schedules.

## Trading Calendar

The study covers **2004-01-01 to 2026-09-01**. The resulting NYSE calendar contains **5,702 trading sessions**, beginning on 2004-01-02 and ending on 2026-09-01. The calendar is generated programmatically and subsequently validated before being passed to the acquisition stage.

```
def get_xnys_trading_dates(start_date: str, end_date: str) -> list[str]:
    calendar = pd_mcal.get_calendar("\acs{NYSE}")
    schedule = calendar.schedule(
        start_date=start_date,
        end_date=end_date,
    )
    return [d.strftime("%Y-%m-%d") for d in schedule.index]
```

| Parameter               | Configuration           |
|-------------------------|-------------------------|
| Exchange                | NYSE                    |
| Study period            | 2004-01-01 → 2026-09-01 |
| First trading session   | 2004-01-02              |
| Last trading session    | 2026-09-01              |
| Trading sessions        | 5,702                   |
| Calendar implementation | pandas_market_calendars |
| Calendar identifier     | NYSE                    |

Table 11.1: NYSE Trading Calendar Configuration

The trading-date sequence forms the pipeline’s **temporal backbone**. The pipeline checks that it is non-empty, ordered, unique, and covers the target span to avoid propagating calendar errors into later API queries.

## Daily Active-Ticker Reconstruction

For every trading session  $t$ , the pipeline queries Massive’s `/v3/reference/tickers` endpoint with the historical date and active-stock filters. The endpoint supports retrieving tickers available on a specified date, filtering by market and active status, with a maximum page size of 1,000 records and pagination through `next_url`.

```
url = "https://api.massive.com/v3/reference/tickers"
params = {
    "date": target_date,
    "active": "true",
    "market": "stocks",
    "limit": 1000,
    "apiKey": self.api_key,
}
```



The resulting daily snapshot can be represented as:

$$S_t = \{i_1, i_2, \dots, i_{N_t}\},$$

where  $S_t$  denotes the set of tickers observed as active on trading session  $t$ , and  $N_t$  is the corresponding number of tickers.

The acquisition procedure for each trading session is:

1. **Select** the next NYSE trading session  $t$ .
2. **Query** Massive for active US stock tickers available on  $t$ .
3. **Follow pagination** until the complete response for that date has been retrieved.
4. **Validate completion** of the entire snapshot rather than partially committing an incomplete result.
5. **Persist** the daily ticker list as an independent raw snapshot.
6. **Record acquisition status** in the corresponding worker manifest.
7. **Proceed** to the next trading session.

This design is important because each daily snapshot is treated as an independent historical observation rather than immediately collapsing the data into a single contemporary ticker list. The raw snapshots therefore preserve the temporal information required to determine when individual securities were observed as active.

The resulting dataset has the conceptual form:

| Date       | Active Tickers       |
|------------|----------------------|
| 2004-01-02 | [A, AA, AACC, ...]   |
| 2004-01-05 | [A, AA, AACC, ...]   |
| 2004-01-06 | [A, AA, ...]         |
|            | ⋮                    |
| 2026-09-01 | [A, AAPL, ABBV, ...] |

A further consideration is that the acquisition is subject to the practical constraints of the available API access. Massive currently documents its Stocks Basic tier as a free tier with a **5 requests/minute** limit, making a multi-decade, date-by-date reconstruction operationally non-trivial. The pipeline therefore treats rate limiting, pagination, retries, checkpointing, and persistent storage as integral parts of the acquisition design rather than as incidental implementation details. These mechanisms are discussed together with the distributed execution strategy in the following section.

## 11.3 Distributed Acquisition and Persistent Storage

The historical reconstruction requires 5,702 trading-session queries, with each date potentially requiring multiple paginated API requests. Under the available free-tier rate constraints, processing the entire period sequentially would make the acquisition unnecessarily slow and vulnerable to interruption. The pipeline therefore distributes the workload across independent workers while maintaining persistent checkpoints so that completed dates do not need to be downloaded again.

### Distributed Execution

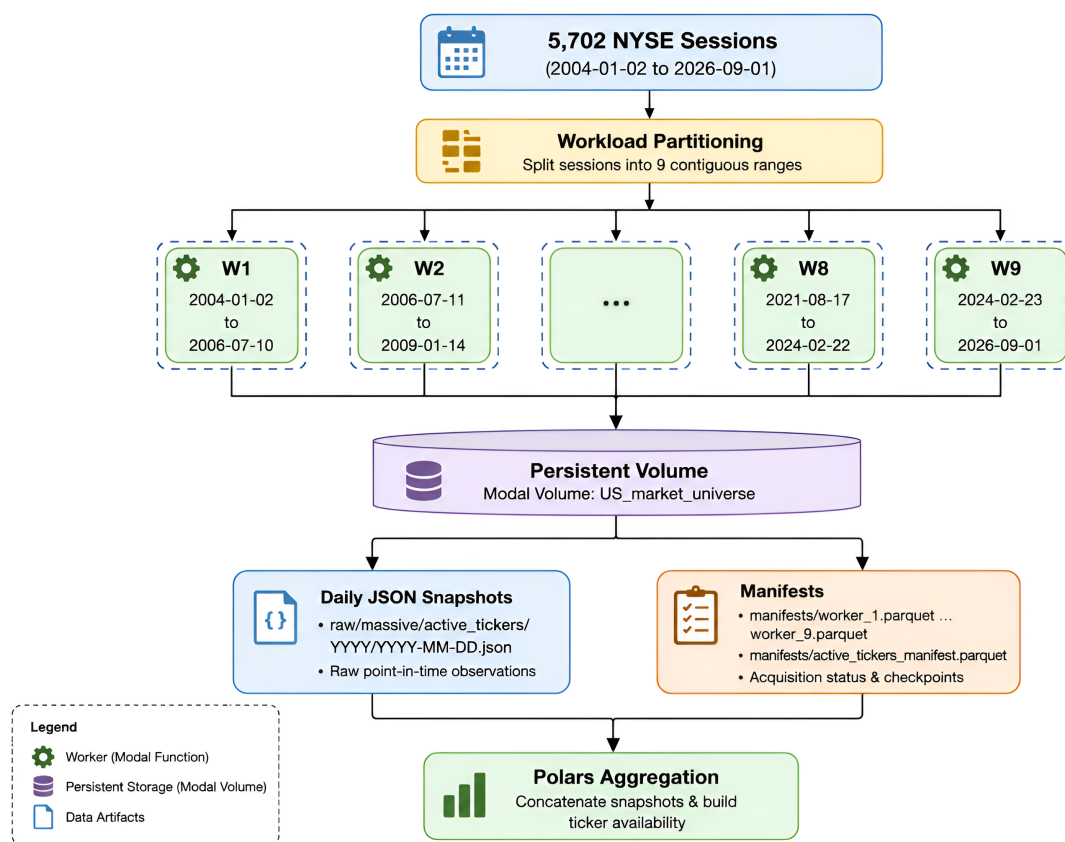


Figure 11.2: Workers Distributed Architecture

The implementation uses Modal to execute the acquisition in parallel. Modal supports remote function execution, configurable retries, secret injection, and persistent Volume storage shared across function instances.

The 5,702 NYSE sessions are initially divided into nine approximately equal chronological partitions, with each worker responsible for a contiguous date range. Contiguous partitioning simplifies progress tracking and makes the resulting manifests directly interpretable.



| Worker       | Sessions     | Initial date range             |
|--------------|--------------|--------------------------------|
| No. 1        | 634          | 2004-01-02 → 2006-07-10        |
| No. 2        | 634          | 2006-07-11 → 2009-01-14        |
| No. 3        | 634          | 2009-01-15 → 2011-07-21        |
| No. 4        | 634          | 2011-07-22 → 2014-01-29        |
| No. 5        | 634          | 2014-01-30 → 2016-08-04        |
| No. 6        | 633          | 2016-08-05 → 2019-02-11        |
| No. 7        | 633          | 2019-02-12 → 2021-08-16        |
| No. 8        | 633          | 2021-08-17 → 2024-02-22        |
| No. 9        | 633          | 2024-02-23 → 2026-09-01        |
| <b>Total</b> | <b>5,702</b> | <b>2004-01-02 → 2026-09-01</b> |

Table 11.2: Parallel NYSE Worker Splits.

Each worker operates independently and uses a dedicated Massive API credential. This isolates rate-limit consumption between workers while allowing the nine partitions to progress concurrently. The workers are spawned simultaneously, after which their execution results are collected by the orchestration process.

The distributed design also incorporates checkpoint-aware workload redistribution. Before assigning work, the orchestration layer examines the persistent manifests to identify dates that have already been completed. Consequently, if a worker terminates after successfully processing part of its assigned interval, subsequent execution does *not* restart the entire interval. Remaining dates can instead be redistributed among available workers. This is particularly important for a long-running acquisition where transient API failures or worker interruptions should not invalidate previously completed work.

## Rate Limiting and Fault Tolerance

Because the acquisition is constrained by API request limits, each worker explicitly controls its request rate. The implementation uses a fixed inter-page delay and exponential-style backoff for transient failures.

| Mechanism              | Configuration                      |
|------------------------|------------------------------------|
| Inter-page delay       | 12 seconds                         |
| Initial retry backoff  | 3 seconds                          |
| Maximum retry backoff  | 30 seconds                         |
| Maximum retries        | 10                                 |
| Completion requirement | Entire daily snapshot must succeed |
| Checkpoint unit        | Individual trading session         |

A daily snapshot is not committed as successful merely because the first API response was received. The worker continues through *all* pagination and only records the date as successfully completed after the complete ticker set has been retrieved. This prevents partially retrieved pages from being interpreted as a valid historical universe.

This distinction is important because an incomplete snapshot would not necessarily generate an obvious runtime error; it could instead silently produce an artificially small historical universe. The acquisition process therefore treats completeness of each date-level snapshot as a prerequisite for persistence.

## Persistent Storage and Checkpointing

All workers access a shared Modal Volume named `US_market_universe`. Modal Volumes provide persistent filesystem-like storage that can be shared by multiple functions and retained across separate executions. Changes must be committed to make them durably available to other containers.

```

/mnt/data/
├── raw/
│   ├── massive/
│   │   ├── active_tickers/
│   │   │   ├── 2004/
│   │   │   │   ├── 2004-01-02.json
│   │   │   │   ├── 2004-01-05.json
│   │   │   │   └── ...
│   │   │   └── 2005/
│   │   │       └── ...
│   └── manifests/
│       ├── worker_1.parquet
│       ├── worker_2.parquet
│       ├── ...
│       ├── worker_9.parquet
│       └── active_tickers_manifest.parquet

```

The separation between raw snapshots and manifests serves two different purposes:



| Artifact              | Purpose                                                         |
|-----------------------|-----------------------------------------------------------------|
| Daily JSON snapshot   | Preserve the actual historical ticker observation               |
| Worker manifest       | Record which dates succeeded or failed                          |
| Consolidated manifest | Provide a global acquisition-status record                      |
| Modal Volume          | Provide persistent shared storage across workers and executions |

The manifests therefore function as the pipeline's checkpoint layer. On a subsequent execution, the system reads the manifests and identifies already completed dates before assigning remaining work. This makes the acquisition resumable and prevents successful API requests from being unnecessarily repeated.

Each raw daily snapshot follows a compact structure:

```
{  
  "date": "YYYY-MM-DD",  
  "tickers": ["AAPL", "MSFT", "NVDA", "..."],  
  "ticker_count": 3667  
}
```

The result is a fault-tolerant acquisition layer rather than a one-shot scraping process. Raw observations remain available independently of worker execution state, while manifests provide sufficient information to resume incomplete portions of the historical period. This persistent intermediate layer is subsequently consumed by the ticker-availability reconstruction described in Section 11.4. Modal's documentation also notes that concurrent modification of the same Volume files should be avoided; accordingly, the implementation assigns separate snapshot files to dates and separate manifest files to workers rather than having multiple workers overwrite the same file.

## 11.4 Ticker Availability Reconstruction

## 11.5 Security-Level OHLCV Acquisition

## 12 Data Integrity

### 12.1 Data Quality and Missing Observations

## 13 Factor Data

To facilitate the systematic risk residualization required for the cross-sectional prediction target, external macroeconomic factor data must be integrated into the pipeline. The daily Fama-French 3-Factor dataset<sup>[6]</sup> is acquired programmatically via `pandas-datareader` directly from the Kenneth R. French Data Library. This dataset provides the daily market risk premium (Mkt-RF), the size factor (SMB), the value factor (HML), and the risk-free rate (RF).

A critical MLOps engineering consideration in acquiring this factor data is chronological alignment and the management of publication lag. Because the Kenneth R. French Data Library operates with an inherent reporting delay relative to live equity markets, the overall observation period of the analytical sample is dynamically truncated to match the maximum available factor date. This strict date intersection prevents the introduction of missing observations into the trailing 126-day rolling OLS regressions and explicitly prohibits the use of forward-filling techniques, which would otherwise introduce forward-looking data leakage into the residualized alpha target.

## 14 Point-in-Time Universe Construction

## Part IV

# Data Transformation

### 14.1 Feature Engineering

#### 14.1.1 Logarithmic Returns

Let  $P_{i,t}$  denote the dividend-adjusted closing price of security  $i$  at time  $t$ . The single-period logarithmic return,  $r_{i,t}$ , is defined as

$$r_{i,t} = \ln \left( \frac{P_{i,t}}{P_{i,t-1}} \right) = \ln(P_{i,t}) - \ln(P_{i,t-1}). \quad (14.1)$$

This transformation represents the continuously compounded return over a single period. Unlike simple returns, logarithmic returns are additive through time, such that the cumulative logarithmic return over multiple periods is equal to the sum of the corresponding single-period returns. This property provides a convenient representation for constructing rolling transformations from sequential price observations.

Because the transformation is based on relative rather than absolute price changes, logarithmic returns are independent of the nominal price level of a security. This provides a comparable representation of price movements across securities with substantially different prices and avoids directly modeling the non-stationary scale of raw price levels.

Within the data-processing pipeline, logarithmic returns therefore provide the baseline representation of short-horizon price dynamics. Subsequent transformations are constructed from this representation where appropriate.

#### 14.1.2 Distance from Simple Moving Average (DSMA)

Let  $SMA_{i,t}^{(w)}$  denote the Simple Moving Average of the closing price of security  $i$  over a rolling window of  $w$  days:

$$SMA_{i,t}^{(w)} = \frac{1}{w} \sum_{k=0}^{w-1} P_{i,t-k}, \quad (14.2)$$

where  $P_{i,t}$  denotes the dividend-adjusted closing price. The Distance from Simple Moving Average (DSMA) measures the proportional deviation of the current price from its recent moving average:

$$DSMA_{i,t}^{(w)} = \frac{P_{i,t} - SMA_{i,t}^{(w)}}{SMA_{i,t}^{(w)}}. \quad (14.3)$$

In this study, the lookback windows are defined as  $w \in \{10, 20, 60\}$  to represent short-, medium-, and long-horizon deviations from the corresponding local price baseline.

DSMA represents the relative position of the current price with respect to its recent moving average. Positive values indicate that the current price is above the corresponding moving average, while negative values indicate that it is below. Expressing the deviation proportionally rather than in absolute price units makes the resulting transformation comparable across securities with different nominal price levels.

As a data transformation, DSMA therefore converts the price level into a normalized measure of relative price position across multiple temporal horizons, providing information complementary to period-to-period returns.

### 14.1.3 Relative Strength Index (RSI)

Let  $\Delta P_{i,t} = P_{i,t} - P_{i,t-1}$  denote the single-period change in the dividend-adjusted closing price of security  $i$ . The price changes are decomposed into positive gains,  $U_{i,t}$ , and absolute losses,  $D_{i,t}$ :

$$U_{i,t} = \max(\Delta P_{i,t}, 0), \quad D_{i,t} = \max(-\Delta P_{i,t}, 0). \quad (14.4)$$

For a lookback window of  $w$  periods, the Relative Strength factor is defined as the ratio of the smoothed average gain to the smoothed average loss:

$$RS_{i,t}^{(w)} = \frac{\text{AvgGain}_{i,t}^{(w)}}{\text{AvgLoss}_{i,t}^{(w)}}. \quad (14.5)$$

The Relative Strength Index is then given by

$$RSI_{i,t}^{(w)} = 100 - \frac{100}{1 + RS_{i,t}^{(w)}}. \quad (14.6)$$

The resulting indicator is bounded on  $[0, 100]$ . Higher values indicate that recent gains have been larger and/or more persistent relative to recent losses, whereas lower values indicate the opposite. The ratio-based construction therefore provides a common numerical representation of recent directional momentum across securities.

Within the data-processing pipeline, RSI transforms sequential price changes into a bounded representation of the relative balance between recent gains and losses. It thus provides information complementary to logarithmic returns and DSMA, which represent individual price changes and relative price position, respectively.

#### 14.1.4 Relative Volume (RVOL)

Let  $V_{i,t}$  denote the raw trading volume of security  $i$  at time  $t$ . Relative Volume,  $RVOL_{i,t}^{(w)}$ , measures current trading activity relative to the security's recent historical volume baseline. The baseline is defined as

$$SMA_{V,i,t}^{(w)} = \frac{1}{w} \sum_{k=0}^{w-1} V_{i,t-k}, \quad (14.7)$$

and Relative Volume is given by

$$RVOL_{i,t}^{(w)} = \frac{V_{i,t}}{SMA_{V,i,t}^{(w)}}. \quad (14.8)$$

A value near 1 indicates that current trading volume is close to its recent average, whereas values substantially above or below 1 indicate unusually high or low activity relative to the security's own historical baseline. By expressing volume relative to its own recent level, the transformation reduces the dependence of the representation on absolute trading volume.

As a data transformation, RVOL converts raw volume into a relative measure of trading intensity. This provides a representation of changes in market activity that is more comparable across securities than absolute volume and complements price-derived transformations by incorporating information from the volume dimension of the OHLCV data.

#### 14.1.5 Ichimoku Indicator

The Ichimoku Kinko Hyo (hereafter, Ichimoku) is a multi-component technical analysis framework that represents price structure across multiple temporal horizons. It combines rolling high–low price ranges into several related reference levels, allowing the relative position and configuration of price to be represented jointly<sup>[4;5]</sup>.

Let  $P_{i,t}^H$ ,  $P_{i,t}^L$ , and  $P_{i,t}^C$  denote the high, low, and closing prices of security  $i$  at time  $t$ . The framework uses three lookback parameters,  $w_1$ ,  $w_2$ , and  $w_3$ .

**Tenkan-sen** The Tenkan-sen ( $TS_{i,t}$ ) is the midpoint of the highest high and lowest low over the short-term window:

$$TS_{i,t} = \frac{\max_{k=0,\dots,w_1-1} P_{i,t-k}^H + \min_{k=0,\dots,w_1-1} P_{i,t-k}^L}{2}. \quad (14.9)$$

It provides a short-horizon reference level derived from the recent trading range and therefore represents local price structure over the corresponding temporal window.

**Kijun-sen** The Kijun-sen ( $KS_{i,t}$ ) applies the same midpoint construction over the longer window  $w_2$ :

$$KS_{i,t} = \frac{\max_{k=0,\dots,w_2-1} P_{i,t-k}^H + \min_{k=0,\dots,w_2-1} P_{i,t-k}^L}{2}. \quad (14.10)$$

Because it is calculated over a longer horizon, the Kijun-sen provides a slower-moving reference for the prevailing price structure and complements the shorter-horizon Tenkan-sen.

**Senkou Span A** Senkou Span A ( $SSA$ ) is defined as the midpoint of the Tenkan-sen and Kijun-sen:

$$SSA_{i,t+w_2} = \frac{TS_{i,t} + KS_{i,t}}{2}. \quad (14.11)$$

It combines short- and medium-horizon reference levels into a single structural representation.

**Senkou Span B** Senkou Span B ( $SSB$ ) is the midpoint of the highest high and lowest low over the longest lookback window:

$$SSB_{i,t+w_2} = \frac{\max_{k=0,\dots,w_3-1} P_{i,t-k}^H + \min_{k=0,\dots,w_3-1} P_{i,t-k}^L}{2}. \quad (14.12)$$

It provides a slower-moving reference based on the longer historical trading range.

**Kumo** The Kumo, or cloud, is the region bounded by Senkou Span A and Senkou Span B. It therefore represents the relationship between the short/medium-horizon structure captured by  $SSA$  and the longer-horizon structure captured by  $SSB$ <sup>[4;5]</sup>.

**Chikou Span** The Chikou Span ( $CS$ ) represents the current closing price plotted  $w_2$  periods backward:

$$CS_{i,t-w_2} = P_{i,t}^C. \quad (14.13)$$

It provides a lagged comparison between current price and historical price levels.

**Temporal Alignment** Because Senkou spans and the Chikou Span are conventionally shifted for visualization, their use in a predictive setting requires point-in-time alignment. Features supplied to the model at time  $t$  must be constructed exclusively from information available no later than  $t$ . A forward-shifted value that depends on observations unavailable at time  $t$  would introduce look-ahead bias. The exact alignment procedure must therefore be defined according to the implemented data pipeline.

Collectively, the Ichimoku components provide a multi-horizon representation of price structure that complements return-, moving-average-, momentum-, and volume-based transformations.

#### 14.1.6 Price Volume Trend (PVT)

Let  $P_{i,t}$  and  $V_{i,t}$  denote the dividend-adjusted closing price and trading volume, respectively, of security  $i$  at time  $t$ . The Price Volume Trend (PVT) is a cumulative price–volume transformation defined recursively as

$$PVT_{i,t} = PVT_{i,t-1} + V_{i,t} \left( \frac{P_{i,t} - P_{i,t-1}}{P_{i,t-1}} \right). \quad (14.14)$$

The change in PVT is therefore proportional to both the relative price movement and the contemporaneous trading volume. Positive price changes increase the cumulative value, whereas negative price changes decrease it. Unlike volume-only transformations, PVT incorporates the magnitude of the price movement into each volume contribution.

As a data transformation, PVT converts the separate price and volume observations into a cumulative price–volume representation. It therefore captures information about the historical interaction between price changes and trading activity that is not represented by logarithmic returns or Relative Volume alone.

Because PVT is cumulative and unbounded, its numerical scale depends on the historical path and volume scale of each security. Any subsequent normalization or transformation should therefore be documented separately from the mathematical definition of PVT.

#### 14.1.7 Rolling Volatility

Let  $r_{i,t}$  denote the single-period logarithmic return of security  $i$  at time  $t$ . Rolling Volatility,  $\sigma_{i,t}^{(w)}$ , measures the dispersion of returns over a historical lookback window of  $w$  periods. The rolling mean return is defined as

$$\bar{r}_{i,t}^{(w)} = \frac{1}{w} \sum_{k=0}^{w-1} r_{i,t-k}, \quad (14.15)$$

and rolling volatility is calculated as the sample standard deviation:

$$\sigma_{i,t}^{(w)} = \sqrt{\frac{1}{w-1} \sum_{k=0}^{w-1} \left( r_{i,t-k} - \bar{r}_{i,t}^{(w)} \right)^2}. \quad (14.16)$$

The resulting quantity represents the historical variability of returns around their local mean. Larger values indicate greater recent return dispersion, whereas smaller values indicate comparatively stable return behavior. Because the calculation is performed on logarithmic

returns rather than absolute price levels, the resulting measure is independent of the nominal price scale of the security.

As a data transformation, rolling volatility summarizes the variability of individual return observations over a defined historical horizon. It therefore complements logarithmic returns by representing the local dispersion of price movements rather than the movement of price itself.

For reproducibility, the implementation must specify the lookback window, whether the volatility measure is retained as a daily standard deviation or annualized, and the degrees-of-freedom convention used in the calculation.

## 14.2 Feature Extraction

To be updated

## 14.3 Feature Selection

To be updated



## Part V

# Alpha Implementation

## 15 Model Architecture

### 15.1 Temporal Encoding (LSTM)

### 15.2 Context Layer (Cross-Sectional Transformer)

### 15.3 Hybrid Loss Function

## 16 Hyperparameter Optimization

## 17 Training Procedure





## Part VI

# Empirical Results

## 18 Descriptive Statistics and Data Diagnostics

### 18.1 Dataset Characteristics

### 18.2 Target and Feature Diagnostics

## 19 Predictive Performance

### 19.1 IC/ICIR

### 19.2 Decile Portfolio Monotonicity

### 19.3 Kendall's Tau Analysis

## 20 Statistical Diagnostics

### 20.1 Statistical Significance

### 20.2 Multiple-Testing and Overfitting Diagnostics

## 21 Ablation Results

### 21.1 Model Architecture Ablation

### 21.2 Loss Function Ablation

### 21.3 Stock ID Embedding Ablation

## 22 Explainability and Regime-Conditioned Analysis

### 22.1 Integrated Gradients Attribution

### 22.2 Feature Importance Across Volatility Regimes

### 22.3 Surrogate Model Fidelity

### 22.4 Interpretation of Model Behavior



## Part VII

# Portfolio Construction



## Part VIII

# Appendixes

## References

- [1] Robert Almgren and Neil Chriss. Optimal execution of portfolio transactions. *Journal of Risk*, 3(2):5–39, 2000. doi: 10.21314/JOR.2000.041.
- [2] David H Bailey and Marcos López de Prado. The deflated sharpe ratio: Correcting for selection bias, backtest overfitting, and non-normality. *The Journal of Portfolio Management*, 40(5):94–107, 2014. doi: 10.3905/jpm.2014.40.5.094.
- [3] Stephen J. Brown, William Goetzmann, Roger G. Ibbotson, and Stephen A. Ross. Survivorship bias in performance studies. *The Review of Financial Studies*, 5(4):553–580, 1992. doi: 10.1093/rfs/5.4.553.
- [4] Yoyo Cahyadi. Ichimoku kinko hyo: Keunikan dan penerapannya dalam strategi perdagangan valuta asing (studi kasus pada pergerakan usd/jpy dan eur/usd) [ichimoku kinko hyo: Uniqueness and its application in foreign exchange trading strategy (case study on usd/jpy and eur/usd movements)]. *Binus Business Review*, 3(1):480–492, 2012. doi: 10.21512/bbr.v3i1.1336.
- [5] Shangkun Deng, Haoran Yu, Chenyang Wei, Tianxiang Yang, and Tatsuro Shimada. The profitability of ichimoku kinkohyo based trading rules in stock markets and fx markets. *International Journal of Finance & Economics*, 26(4):5321–5336, 2021. doi: 10.1002/ijfe.2067.
- [6] Eugene F. Fama and Kenneth R. French. The cross-section of expected stock returns. *The Journal of Finance*, 47(2):427–465, 1992. doi: 10.1111/j.1540-6261.1992.tb04398.x.
- [7] Eugene F. Fama and Kenneth R. French. Common risk factors in the returns on stocks and bonds. *Journal of Financial Economics*, 33(1):3–56, 1993. doi: 10.1016/0304-405X(93)90023-5.
- [8] Thomas Fischer and Christopher Krauss. Deep learning with long short-term memory networks for financial market predictions. *European Journal of Operational Research*, 270(2):654–669, 2018. doi: 10.1016/j.ejor.2017.11.054.
- [9] Shihao Gu, Bryan Kelly, and Dacheng Xiu. Empirical asset pricing via machine learning. *The Review of Financial Studies*, 33(5):2223–2273, 2020. doi: 10.1093/rfs/hhaa009.
- [10] Daniel Poh, Bryan Lim, Stefan Zohren, and Stephen Roberts. Building cross-sectional systematic strategies by learning to rank. *The Journal of Financial Data Science*, 3(2):70–86, 2021. doi: 10.3905/jfds.2021.1.060.



- [11] Leonard J. Tashman. Out-of-sample tests of forecasting accuracy: An analysis and review. *International Journal of Forecasting*, 16(4):437–450, 2000. doi: 10.1016/S0169-2070(00)00065-0.
- [12] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.